

lecture_07

August 8, 2026

1 Lecture 7: Simplex Method

{note} In Lecture 6, we handed our LP formulations to `scipy.optimize.linprog(method='highs')` and let it return an optimal solution. This lecture opens that black box. We derive the **standard form** of an LP, show that its basic feasible solutions correspond exactly to the corner points we traced graphically in Lecture 4, and then develop the **Simplex Algorithm** - the pivoting procedure that HiGHS, Excel Solver, and virtually every LP solver in the world use (in some refined form) to move efficiently from corner point to corner point until the optimum is reached.

Learning Objectives

By the end of this notebook, you will be able to: 1. Convert a general LP into standard form by introducing slack variables, and express it in matrix form $\mathbf{Ax} = \mathbf{b}$. 2. Explain why basic feasible solutions (BFS) of the standard form correspond exactly to corner points of the feasible region. 3. Execute the Simplex Algorithm on in-class and take-away exercises.

Prerequisites: LP Formulation (Lecture 3), Graphical Method (Lecture 4), Python SciPy Method (Lecture 6)

Estimated time: 90 minutes (including in-class exercises)

1.1 The Simplex Method

1.1.1 General Standard Form

A typical linear optimization problem with n decision variables and m constraints can be expressed as,

$$\max_{\mathbf{x}} \quad f(\mathbf{x}) = c_1x_1 + \dots + c_jx_j + \dots + c_nx_n$$

Subject to:

$$\begin{aligned}
a_{11}x_1 \cdots + a_{1j}x_j \cdots + a_{1n}x_n &\leq b_1 \\
&\vdots \\
a_{i1}x_1 \cdots + a_{ij}x_j \cdots + a_{in}x_n &\leq b_i \\
&\vdots \\
a_{m1}x_1 \cdots + a_{mj}x_j \cdots + a_{mn}x_n &\leq b_m \\
x_j &\geq 0 \quad \forall j \in \{1, \dots, n\}
\end{aligned}$$

Here z is the objective function, x_j is a decision variable, c_j is the objective coefficient of x_j , a_{ij} is the coefficient of x_j in constraint i , and b_i is the right-hand-side (RHS) of constraint i .

1.1.2 Slack Standard Form

To apply the Simplex Algorithm, each $\leq$ inequality is converted to an equality by adding a non-negative **slack variable** s_i :

$$\begin{aligned}
a_{11}x_1 \cdots + a_{1j}x_j \cdots + a_{1n}x_n + s_1 &= b_1 \\
&\vdots \\
a_{i1}x_1 \cdots + a_{ij}x_j \cdots + a_{in}x_n + s_i &= b_i \\
&\vdots \\
a_{m1}x_1 \cdots + a_{mj}x_j \cdots + a_{mn}x_n + s_m &= b_m \\
s_i &\geq 0 \quad \forall i \in \{1, \dots, m\} \\
x_j &\geq 0 \quad \forall j \in \{1, \dots, n\}
\end{aligned}$$

Each slack variable s_i absorbs the unused capacity of constraint i : if constraint i is not binding at a solution, $s_i > 0$; if it is binding, $s_i = 0$.

1.1.3 Matrix Form

Defining the full variable vector $\tilde{\mathbf{x}} = [x_1, \dots, x_n, s_1, \dots, s_m]^\top$, the standard form can be written compactly as,

$$\begin{aligned}
\max_{\tilde{\mathbf{x}}} \quad & z = \mathbf{c}^\top \mathbf{x} \\
\text{subject to} \quad & [\mathbf{A} \mid \mathbf{I}] \tilde{\mathbf{x}} = \mathbf{b} \\
& \tilde{\mathbf{x}} \geq \mathbf{0}
\end{aligned}$$

where,

$$\mathbf{A} = \begin{bmatrix} a_{11} & \cdots & a_{1j} & \cdots & a_{1n} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ a_{i1} & \cdots & a_{ij} & \cdots & a_{in} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ a_{m1} & \cdots & a_{mj} & \cdots & a_{mn} \end{bmatrix}_{m \times n} \quad \mathbf{b} = \begin{bmatrix} b_1 \\ \vdots \\ b_i \\ \vdots \\ b_m \end{bmatrix}_{m \times 1} \quad \mathbf{c} = \begin{bmatrix} c_1 \\ \vdots \\ c_j \\ \vdots \\ c_n \end{bmatrix}_{n \times 1} \quad \tilde{\mathbf{x}} = \begin{bmatrix} x_1 \\ \vdots \\ x_n \\ s_1 \\ \vdots \\ s_m \end{bmatrix}_{(n+m) \times 1}$$

Here $[\mathbf{A} \mid \mathbf{I}]$ is the $m \times (n + m)$ **augmented matrix**, formed by appending the $m \times m$ identity matrix $\mathbf{I}$ — one column per slack variable — to the constraint matrix $\mathbf{A}$.

1.1.4 Basis Matrix and Basic Feasible Solution

The augmented system $[\mathbf{A} \mid \mathbf{I}] \tilde{\mathbf{x}} = \mathbf{b}$ has $n + m$ unknowns but only m equations, so it has infinitely many solutions. The Simplex Algorithm navigates this space by working with basic feasible solutions.

Let $\mathcal{B} \subseteq \{1, 2, \dots, n + m\}$ be any set of m column indices of $[\mathbf{A} \mid \mathbf{I}]$, and let $\mathbf{B}$ be the $m \times m$ sub-matrix formed by those columns. If $\mathbf{B}$ is invertible, then $\mathcal{B}$ is called a **basis** and $\mathbf{B}$ is the **basis matrix**. Setting all n variables outside $\mathcal{B}$ (**non-basic variables**, NBV) to zero and solving for the remaining m variables (**basic variables**, BV) gives the **basic solution**:

$$\mathbf{x}_B = \mathbf{B}^{-1} \mathbf{b}$$

If additionally $\mathbf{x}_B \geq \mathbf{0}$, the solution is a **basic feasible solution (BFS)**.

1.1.5 Simplex Algorithm — BFS Enumeration

With $n + m$ variables and m equations, there are at most $\binom{n+m}{m}$ candidate bases. The Simplex Algorithm (in its BFS enumeration form) proceeds as:

1. Convert the LP to standard form, constructing the augmented matrix $[\mathbf{A} \mid \mathbf{I}]$.
2. Enumerate all $\binom{n+m}{m}$ choices of m columns to form a candidate basis matrix $\mathbf{B}$.
3. For each candidate, check whether $\mathbf{B}$ is invertible. If not, skip.
4. Compute $\mathbf{x}_B = \mathbf{B}^{-1} \mathbf{b}$. If $\mathbf{x}_B \geq \mathbf{0}$, record it as a BFS.
5. Evaluate the objective at each BFS: $z = \mathbf{c}_B^\top \mathbf{x}_B$, where $\mathbf{c}_B$ is the sub-vector of objective coefficients for the basic variables.
6. Select the BFS with the highest z as the optimal solution.

`{tip}` Each BFS corresponds to a **corner point** of the feasible region — and vice versa. This is why the Simplex Algorithm only ever needs to examine corner points, not the entire (infinite) feasible region.

Checking all $\binom{n+m}{m}$ combinations is tractable for small problems (as we will do in the in-class exercise below), but becomes computationally intensive for real transportation models with hundreds of variables — fleet schedules, network flows, signal timings. More advanced implementations therefore use the **Simplex Tableau**, which avoids full enumeration:

1. Start at one easy-to-find BFS (the origin, where all slack variables are basic).

2. Check whether moving to an adjacent BFS (differing in exactly one basic/non-basic swap) improves the objective.
3. If yes, move there — this is called a pivot.
4. Repeat until no adjacent BFS improves the objective — that BFS is optimal.

Geometrically, this is identical to the graphical method (Lecture 4): starting at a corner point and moving along edges of the feasible region towards better corners. The simplex tableau is simply the algebraic bookkeeping device that performs this walk in n dimensions, where we cannot draw a picture.

1.2 In-Class Exercise

1.2.1 Exercise 1

The Chennai Metropolitan Transport Corporation (MTC) is evaluating two new express services on a feeder corridor:

Service	Net contribution margin ('000/trip)
E ₁ — Premium Express	4
E ₂ — Limited-Stop Express	3

Two resources constrain how many trips of each service MTC can run per day:

- Driver-shift hours: both services require 1 driver-shift-hour per trip; 40 driver-shift-hours are available per day.
- Fuel allocation: E₁ (longer route) consumes 2 units of fuel per trip, E₂ consumes 1 unit; the daily fuel budget is 60 units.

Let $x_1, x_2 \geq 0$ denote daily trips of E₁ and E₂; maximize the total contribution.

Objective:

$$\max_{x_1, x_2} z = 4x_1 + 3x_2$$

Subject to:

$$\begin{aligned} x_1 + x_2 &\leq 40 && \text{(driver-shift hours)} \\ 2x_1 + x_2 &\leq 60 && \text{(fuel budget)} \\ x_1, x_2 &\geq 0 \end{aligned}$$

Standard Formulation Objective:

$$\max_{x_1, x_2} z = 4x_1 + 3x_2$$

Subject to:

$$\begin{aligned}
x_1 + x_2 + s_1 &= 40 \\
2x_1 + x_2 + s_2 &= 60 \\
x_1, x_2, s_1, s_2 &\geq 0
\end{aligned}$$

We now have $n + m = 2 + 2 = 4$ variables and only $m = 2$ equations — the system has infinitely many solutions. The simplex method navigates exactly these solutions.

Basic Feasible Solution With $n + m = 4$ variables and $m = 2$ equations, we can freely set $n = 2$ variables to zero (the non-basic variables, NBV) and solve the remaining $m = 2$ equations uniquely for the basic variables (BV). If the resulting values satisfy $\mathbf{x}, \mathbf{s} \geq 0$, the solution is a basic feasible solution (BFS).

There are $\binom{n+m}{m} = \binom{4}{2} = 6$ ways to choose which 2 variables are non-basic. Enumerating all of them for In-Class Exercise 1:

Non-Basic (= 0)	Basic	(x_1, x_2)	(s_1, s_2)	Feasible?	$z = 4x_1 + 3x_2$
x_1, x_2	s_1, s_2	(0, 0)	(40, 60)	Yes	0
x_2, s_2	x_1, s_1	(30, 0)	(10, 0)	Yes	120
x_2, s_1	x_1, s_2	(40, 0)	(0, -20)	No	—
x_1, s_2	x_2, s_1	(0, 60)	(-20, 0)	No	—
x_1, s_1	x_2, s_2	(0, 40)	(0, 20)	Yes	120
s_1, s_2	x_1, x_2	(20, 20)	(0, 0)	Yes	140

{tip} Plot the four **feasible** (x_1, x_2) pairs above - $(0,0)$, $(30,0)$, $(0,40)$, and $(20,20)$ - on graph paper, or recall the feasible-region plots from Lecture 4. They are exactly the **corner points** of the feasible region! Every BFS corresponds to a corner point, and every corner point corresponds to a BFS. The optimum, $z^* = 140$ at $(x_1, x_2) = (20, 20)$, is - as the graphical method already told us - a corner point.

This equivalence is the foundation of the simplex method: rather than searching the entire (infinite) feasible region, we only ever need to examine its corner points (BFS).

1.3 Take-Away Exercise

Solve the following using the Simplex Method and infer the results.

1.3.1 Exercise 1

A Lucknow-based freight operator must decide how many vehicle-trips of three types — Type A (x_1), Type B (x_2), Type C (x_3) — to deploy today, earning net profits of 50, 40, and 30 (in '00/trip) respectively.

Three resources constrain the day's operations:

- Fuel: 3, 1, 2 units/trip for Types A, B, C; budget = 24 units.
- Loading-dock hours: 1, 2, 1 hours/trip; available = 16 hours.

- Driver shifts: 1, 1, 1 shift/trip; available = 10 shifts.

Maximize net profits.

1.3.2 Exercise 2

A Mumbai-based logistics firm dispatches two container types from its port yard each day: 20-ft containers (x_1) and 40-ft containers (x_2), earning net margins of 8,000 and 5,000 per dispatch (in '000: 8 and 5).

Two yard resources constrain daily dispatches:

- Crane-hours: 4 hours/dispatch for 20-ft, 2 hours/dispatch for 40-ft; 60 crane-hours/day available.
- Gate-clearance hours: 2 hours/dispatch for 20-ft, 4 hours/dispatch for 40-ft; 48 gate-clearance-hours/day available.

Maximize net margins.

1.3.3 Exercise 3

An e-commerce company allocates its delivery fleet across three Bengaluru zones — Whitefield (x_1), Electronic City (x_2), and Koramangala (x_3). Net revenue per vehicle-day is 3,000, 5,000, and 4,000 (in '000: 3, 5, 4) respectively. Each zone's road network and parking can support only a limited number of delivery vehicles (Whitefield: 18, Electronic City: 12, Koramangala: 9) and the company has a limited fleet of 30 vehicles.

Maximize net revenue.

Connection to Lectures 4 and 6: the simplex method is the algebraic twin of the graphical method (Lecture 4) — both walk corner points of the feasible region — and it is precisely what `scipy.optimize.linprog(method='highs')` (Lecture 6) executes internally (in a far more numerically robust form, using revised-simplex or interior-point variants for large problems).

1.4 Further Reading

- Vanderbei, R.J. (2014). *Linear Programming: Foundations and Extensions* — rigorous treatment of the simplex method, degeneracy, and cycling.
- Bertsimas, D. and Tsitsiklis, J.N. (1997). *Introduction to Linear Optimization* — geometric and algebraic perspectives on the simplex method.
- SciPy documentation: `scipy.optimize.linprog` (`method='highs'`) — [docs.scipy.org](https://docs.scipy.org/doc/scipy/reference/optimize.linprog.html)